Sorting Algorithms Pseudocode

1. Bubble Sort

Algorithm 1 Bubble Sort

```
1:  procedure BubbleSort(A)
2:      n ← length(A)
3:      for i ← 0 to n − 1 do
4:          for j ← 0 to n − i − 2 do
5:              if A[j] > A[j + 1] then
6:                  swap A[j] and A[j + 1]
7:              end if
8:          end for
9:      end for
10: end procedure
```

2. Selection Sort

Algorithm 2 Selection Sort

```
1:  procedure SelectionSort(A)
2:      n ← length(A)
3:      for i ← 0 to n − 2 do
4:          min_idx ← i
5:          for j ← i + 1 to n − 1 do
6:              if A[j] < A[min_idx] then
7:                  min_idx ← j
8:              end if
9:          end for
10:         swap A[i] and A[min_idx]
11:     end for
12: end procedure
```

3. Insertion Sort

Algorithm 3 Insertion Sort

```
1: procedure InsertionSort(A)
2:     n ← length(A)
3:     for i ← 1 to n − 1 do
4:         key ← A[i]
5:         j ← i − 1
6:         while j ≥ 0 and A[j] > key do
7:             A[j + 1] ← A[j]
8:             j ← j − 1
9:         end while
10:        A[j + 1] ← key
11:    end for
12: end procedure
```

4. Counting Sort

Algorithm 4 Counting Sort

```
1: procedure CountingSort(A, B, k)
2:                                  ▷ A: input array, B: output array, k: max value
3:     let C[0 . . . k] be a new array initialized to 0
4:     for j ← 1 to length(A) do
5:         C[A[j]] ← C[A[j]] + 1
6:     end for
7:     for i ← 1 to k do
8:         C[i] ← C[i] + C[i − 1]
9:     end for
10:    for j ← length(A) downto 1 do
11:        B[C[A[j]]] ← A[j]
12:        C[A[j]] ← C[A[j]] − 1
13:    end for
14: end procedure
```

5. Merge Sort

Algorithm 5 Merge Sort

```
1: procedure MergeSort(A, p, r)
2:     if p < r then
3:         q ← ⌊(p + r)/2⌋
4:         MergeSort(A, p, q)
5:         MergeSort(A, q + 1, r)
6:         Merge(A, p, q, r)
7:     end if
8: end procedure
```

Algorithm 6 Merge Procedure

```
1: procedure Merge(A, p, q, r)
2:     n_1 ← q − p + 1
3:     n_2 ← r − q
4:     let L[1 . . . n_1 + 1] and R[1 . . . n_2 + 1] be new arrays
5:     for i ← 1 to n_1 do
6:         L[i] ← A[p + i − 1]
7:     end for
8:     for j ← 1 to n_2 do
9:         R[j] ← A[q + j]
10:    end for
11:    L[n_1 + 1] ← ∞
12:    R[n_2 + 1] ← ∞
13:    i ← 1
14:    j ← 1
15:    for k ← p to r do
16:        if L[i] ≤ R[j] then
17:            A[k] ← L[i]
18:            i ← i + 1
19:        else
20:            A[k] ← R[j]
21:            j ← j + 1
22:        end if
23:    end for
24: end procedure
```

6. Quick Sort

Algorithm 7 Quick Sort

```
1: procedure QuickSort(A, p, r)
2:     if p < r then
3:         q ← Partition(A, p, r)
4:         QuickSort(A, p, q − 1)
5:         QuickSort(A, q + 1, r)
6:     end if
7: end procedure
```

Algorithm 8 Partition Procedure

```
1: procedure Partition(A, p, r)
2:     x ← A[r]
3:     i ← p − 1
4:     for j ← p to r − 1 do
5:         if A[j] ≤ x then
6:             i ← i + 1
7:             swap A[i] and A[j]
8:         end if
9:     end for
10:    swap A[i + 1] and A[r]
11:    return i + 1
12: end procedure
```